

## 1. Print a string using command line arguments

```
public class PrintString {
    public static void main(String[] args) {
        if (args.length > 0) {
            StringBuilder sb = new StringBuilder();
            for (String s : args) {
                sb.append(s).append(" ");
            }
            System.out.println(sb.toString().trim());
        } else {
            System.out.println("Java is an Object Oriented
Programming");
        }
    }
}
```

### Output:

Java is an Object Oriented Programming

## 2. Class Box, subclass BoxWeight to find weight of the box

```
class Box {
    double width, height, depth;

    Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }

    double volume() {
        return width * height * depth;
    }
}

class BoxWeight extends Box {
    double weight;

    BoxWeight(double w, double h, double d, double wt) {
        super(w, h, d);
        weight = wt;
    }

    void display() {
        System.out.println("Volume: " + volume());
        System.out.println("Weight: " + weight);
    }
}

public class BoxWeightDemo {
    public static void main(String[] args) {
        BoxWeight b = new BoxWeight(10, 5, 4, 25.5);
        b.display();
    }
}
```

### Output:

Volume: 200.0  
Weight: 25.5

### 3. Area & Perimeter of Rectangle, Triangle, Circle (Constructor overloading + Inheritance)

```
class Shape {
    String name;

    Shape() {
        name = "Shape";
    }

    Shape(String name) {
        this.name = name;
    }

    double area() {
        return 0;
    }

    double perimeter() {
        return 0;
    }
}

class Rectangle extends Shape {
    double length, breadth;

    Rectangle(double length, double breadth) {
        super("Rectangle");
        this.length = length;
        this.breadth = breadth;
    }

    double area() {
        return length * breadth;
    }

    double perimeter() {
        return 2 * (length + breadth);
    }
}

class Triangle extends Shape {
    double a, b, c;

    Triangle(double a, double b, double c) {
        super("Triangle");
        this.a = a;
        this.b = b;
        this.c = c;
    }

    double area() {
        double s = (a + b + c) / 2;
        return Math.sqrt(s * (s - a) * (s - b) * (s - c));
    }

    double perimeter() {
        return a + b + c;
    }
}
```

```
class Circle extends Shape {
    double radius;

    Circle(double radius) {
        super("Circle");
        this.radius = radius;
    }

    double area() {
        return Math.PI * radius * radius;
    }

    double perimeter() {
        return 2 * Math.PI * radius;
    }
}

public class ShapeDemo {
    public static void main(String[] args) {
        Shape[] shapes = {
            new Rectangle(4, 5),
            new Triangle(3, 4, 5),
            new Circle(7)
        };

        for (Shape s : shapes) {
            System.out.println(s.name + " -> Area: " + s.area() + ",
Perimeter: " + s.perimeter());
        }
    }
}
```

### **Output:**

```
Rectangle -> Area: 20.0, Perimeter: 18.0
Triangle -> Area: 6.0, Perimeter: 12.0
Circle -> Area: 153.93804002589985, Perimeter: 43.982297150257104
```

## 4. Demo1, Demo2 (subclass of Demo1), Demo3 (subclass of Demo2) – Method Overloading & Overriding

```
class Demo1 {
    void show() {
        System.out.println("show() in Demo1");
    }

    void show(String msg) {
        System.out.println("show(String) in Demo1: " + msg);
    }
}

class Demo2 extends Demo1 {
    void show(int num) {
        System.out.println("show(int) in Demo2 (overloading): " +
num);
    }
}

class Demo3 extends Demo2 {
    @Override
    void show() {
        System.out.println("show() overridden in Demo3");
    }
}

public class OverloadOverrideDemo {
    public static void main(String[] args) {
        Demo3 d = new Demo3();
        d.show();           // overridden version
        d.show("Hello");    // inherited from Demo1
        d.show(10);         // overloaded in Demo2
    }
}
```

### Output:

```
show() overridden in Demo3
show(String) in Demo1: Hello
show(int) in Demo2 (overloading): 10
```

## 5. Interface Marks with total\_marks(), class Student

```
interface Marks {
    int total_marks();
}

class Student implements Marks {
    int m1, m2, m3;

    Student(int m1, int m2, int m3) {
        this.m1 = m1;
        this.m2 = m2;
        this.m3 = m3;
    }

    public int total_marks() {
        return m1 + m2 + m3;
    }
}

public class StudentMarksDemo {
    public static void main(String[] args) {
        Student s = new Student(80, 75, 90);
        System.out.println("Total Marks: " + s.total_marks());
    }
}
```

### Output:

Total Marks: 245

## 6. Class Overload – Method Overloading of test()

```
class Overload {
    void test() {
        System.out.println("test() - no arguments");
    }

    void test(int a) {
        System.out.println("test(int): " + a);
    }

    void test(int a, int b) {
        System.out.println("test(int, int): " + (a + b));
    }

    double test(double a) {
        System.out.println("test(double): " + a);
        return a * a;
    }
}

public class OverloadDemo {
    public static void main(String[] args) {
        Overload obj = new Overload();
        obj.test();
        obj.test(5);
        obj.test(5, 10);
        System.out.println("Result: " + obj.test(2.5));
    }
}
```

### Output:

```
test() - no arguments
test(int): 5
test(int, int): 15
test(double): 2.5
Result: 6.25
```

## 7. Class Box – Constructor Overloading

```
class Box {
    double width, height, depth;

    Box() {
        width = height = depth = 0;
    }

    Box(double side) {
        width = height = depth = side;
    }

    Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }

    double volume() {
        return width * height * depth;
    }
}

public class BoxConstructorDemo {
    public static void main(String[] args) {
        Box b1 = new Box();
        Box b2 = new Box(7);
        Box b3 = new Box(3, 4, 5);

        System.out.println("b1 volume: " + b1.volume());
        System.out.println("b2 volume: " + b2.volume());
        System.out.println("b3 volume: " + b3.volume());
    }
}
```

### Output:

```
b1 volume: 0.0
b2 volume: 343.0
b3 volume: 60.0
```

## 8. Class Box, subclass BoxWeight demonstrating `super`

```
class Box {
    double width, height, depth;

    Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }

    double volume() {
        return width * height * depth;
    }
}

class BoxWeight extends Box {
    double weight;

    BoxWeight(double w, double h, double d, double weight) {
        super(w, h, d); // calling parent constructor
        this.weight = weight;
    }

    void display() {
        System.out.println("Volume (from super): " + super.volume());
        System.out.println("Weight: " + weight);
    }
}

public class SuperDemo {
    public static void main(String[] args) {
        BoxWeight bw = new BoxWeight(2, 3, 4, 15.0);
        bw.display();
    }
}
```

### Output:

```
Volume (from super): 24.0
Weight: 15.0
```

## 9. Box, BoxWeight, Shipment – Multilevel Inheritance

```
class Box {
    double width, height, depth;

    Box(double w, double h, double d) {
        width = w;
        height = h;
        depth = d;
    }

    double volume() {
        return width * height * depth;
    }
}

class BoxWeight extends Box {
    double weight;

    BoxWeight(double w, double h, double d, double weight) {
        super(w, h, d);
        this.weight = weight;
    }
}

class Shipment extends BoxWeight {
    double cost;

    Shipment(double w, double h, double d, double weight, double cost)
    {
        super(w, h, d, weight);
        this.cost = cost;
    }

    void display() {
        System.out.println("Volume: " + volume());
        System.out.println("Weight: " + weight);
        System.out.println("Shipping Cost: " + cost);
    }
}

public class MultilevelInheritanceDemo {
    public static void main(String[] args) {
        Shipment s = new Shipment(5, 6, 7, 20, 150.75);
        s.display();
    }
}
```

### Output:

```
Volume: 210.0
Weight: 20.0
Shipping Cost: 150.75
```

## 10. Dynamic Method Dispatch

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Cat meows");
    }
}

public class DynamicDispatchDemo {
    public static void main(String[] args) {
        Animal a;    // reference of parent class

        a = new Dog();
        a.sound();    // calls Dog's sound() at runtime

        a = new Cat();
        a.sound();    // calls Cat's sound() at runtime
    }
}
```

### Output:

```
Dog barks
Cat meows
```

## 11. Figure, Rectangle, Triangle – Runtime Polymorphism

```
class Figure {
    double dim1, dim2;

    Figure(double a, double b) {
        dim1 = a;
        dim2 = b;
    }

    double area() {
        System.out.println("Area for Figure is undefined");
        return 0;
    }
}

class Rectangle extends Figure {
    Rectangle(double a, double b) {
        super(a, b);
    }

    @Override
    double area() {
        return dim1 * dim2;
    }
}

class Triangle extends Figure {
    Triangle(double a, double b) {
        super(a, b);
    }

    @Override
    double area() {
        return 0.5 * dim1 * dim2;
    }
}

public class RuntimePolymorphismDemo {
    public static void main(String[] args) {
        Figure f;

        f = new Rectangle(9, 5);
        System.out.println("Rectangle area: " + f.area());

        f = new Triangle(10, 8);
        System.out.println("Triangle area: " + f.area());
    }
}
```

### Output:

```
Rectangle area: 45.0
Triangle area: 40.0
```

## 12. Abstract Class Demonstration

```
abstract class Vehicle {
    abstract void start();

    void fuelType() {
        System.out.println("Generic fuel type");
    }
}

class Car extends Vehicle {
    @Override
    void start() {
        System.out.println("Car starts with a key/button");
    }
}

class Bike extends Vehicle {
    @Override
    void start() {
        System.out.println("Bike starts with a kick/self-start");
    }
}

public class AbstractClassDemo {
    public static void main(String[] args) {
        Vehicle v1 = new Car();
        Vehicle v2 = new Bike();

        v1.start();
        v2.start();
        v1.fuelType();
    }
}
```

### Output:

```
Car starts with a key/button
Bike starts with a kick/self-start
Generic fuel type
```